

QSPI Flash XIP Controller Integration for Croc

User-domain memory-mapped Quad-SPI controller with direct boot support

Jan Häussermann Nino Singer

June 2026

Executive Summary

This project extends the CROC reference flow by replacing the default user-domain error target with a usable external-flash subsystem. The implemented design adds a memory-mapped QSPI execute-in-place (XIP) window, a custom OBI-to-AHB-LITE bridge, the integrated Efabless flash-read controller and direct-mapped cache, a simulation flash model, and the software and boot changes required to execute code from the user domain. The integration keeps the original Croc-domain architecture intact: the CVE2 core accesses the new subsystem through ordinary OBI transactions, and no dedicated fetch path was added to the CPU.

The final user-domain map is split into two functional regions. The first 4 KiB at 0x2000_0000 hold a small user ROM containing the required group identification string. The second region starts at 0x2000_1000 and maps a 16 MiB QSPI/XIP window whose byte offset zero corresponds to flash byte address zero. Software is linked so that the vector table, startup code, text, and read-only data reside in the XIP region, while writable data and the stack remain in on-chip SRAM.

Fresh Verilator regressions were completed using the dedicated QSPI programs. Both the sample-read test and the XIP-versus-SRAM test reached **Simulation finished: SUCCESS**. The sample-read regression compared eleven addresses across the 16 MiB XIP window against the generated `.flash.hex` image; all eleven observed words matched. The XIP-versus-SRAM regression showed non-zero program words in the XIP window and zero values at the sampled SRAM locations. **The simulation flow commands differ from the reference flow and can be found in subsection 5.1 Simulation Flow.**

The final QSPI-integrated design was synthesized, placed, routed, converted to GDS, and checked with the available signoff flow. Calibre LVS passed with the overall comparison result **CORRECT**. KLayout DRC reported only `metal1_pin_Offgrid` markers located inside the supplied SRAM and pad macro GDS views. According to the course FAQ, these known macro-library markers may be waived because they are not physical manufacturing shapes introduced by the project.

At the 100 MHz target frequency, the extension introduces a setup WNS of -0.64 ns with 2758 setup violations, indicating that the maximum clock frequency may need to be slightly reduced. Hold timing passes with zero hold violations. The remaining electrical checks report 189 maximum-slew violations, 2 maximum-fanout violations, and 83 maximum-capacitance violations. According to the supervisors, these results are considered usual and expected, and they are comparable to those observed in the reference-flow design. With adjustments in the backend flow, the maximum-slew, maximum-fanout, and maximum-capacitance violations were even partially reduced compared to the reference flow.

1 Project Context and Design Targets

The reference flow exposes a user-domain address range but, in its original state, terminates accesses with an OBI error subordinate. For this project, the objective was to turn that region into a useful non-volatile memory interface suitable for software execution and data reads without disturbing the baseline Croc-domain structure, the existing peripheral map, or the CVE2 interface.

The implemented scope is intentionally narrow and well-defined:

- a read-only memory-mapped QSPI/XIP window starting at 0x2000_1000;
- a 4 KiB user ROM at 0x2000_0000 for the required ASCII group string;
- a direct boot hand-off from the Boot ROM to the XIP base address;
- external pin routing for QSPI through GPIO26–GPIO31;
- a simulation-only flash memory model for Verilator;
- build and linker updates so generated software is placed in flash and emitted as byte-per-line `.flash.hex`.

The design deliberately does *not* implement flash program or erase support, a general-purpose AHB-LITE subordinate interface, write transactions into external flash, a configurable boot-source selection mechanism, or power estimation. The project also does not claim setup-timing at the 100 MHz implementation target.

Starting Point in the Reference Flow

In the unmodified reference flow, the user domain contains no manager-side logic and returns 0xBADCAB1E for user accesses that hit the error subordinate. The CROC memory map already reserves 0x2000_0000 and above for the user domain, with 0x2000_0000..0x2000_0FFF recommended for a user ROM string. The present project follows this recommendation and uses the next 16 MiB window for the QSPI/XIP data path.

Top-Level Deliverables

The deliverables relevant to the report are grouped into four categories:

1. RTL modifications under `rtl/`, especially `user_domain.sv`, `user_pkg.sv`, the new bridge, top-level SoC/chip routing, and the Boot ROM.
2. Verification infrastructure under `rtl/test/` and `verilator/`, including the flash model, testbench modifications, and execution scripts.
3. Software support under `sw/`, including the linker changes, generated `.flash.hex` images, and dedicated QSPI tests.
4. Backend and signoff artifacts under `yosys/`, `openroad/`, and `klayout/`.

2 Architecture and Address Map

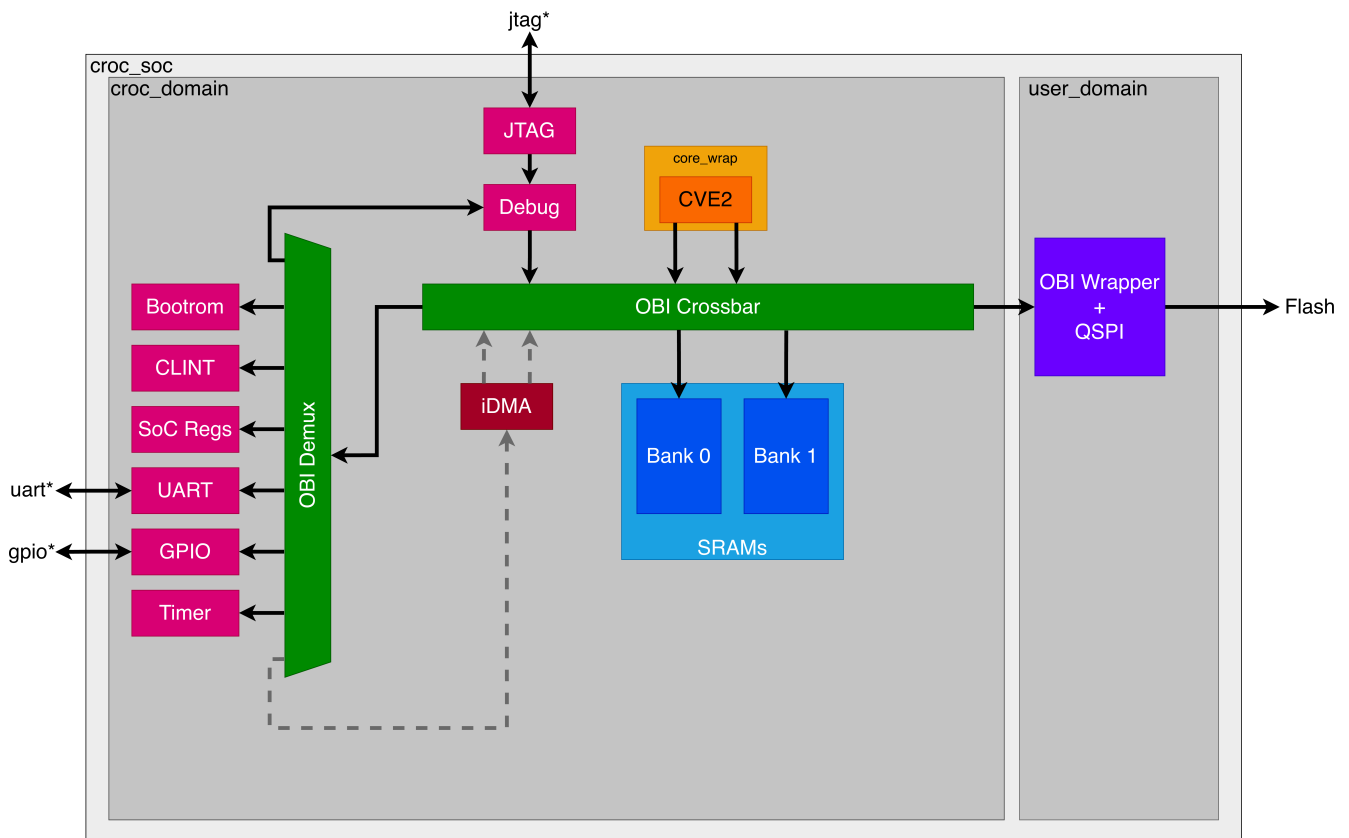


Figure 1: Project block diagram. The user domain now contains the OBI wrapper and QSPI controller path.

The new subsystem is located entirely in the user domain. The existing Croc OBI crossbar forwards accesses in the user address range into `user_domain.sv`. Inside that block, a second address decode selects between a small ROM, the new QSPI path, and the original error responder. The CPU therefore sees the QSPI subsystem as a standard memory-mapped address region, which means both instruction fetches and ordinary loads use the same path.

2.1 Address Map

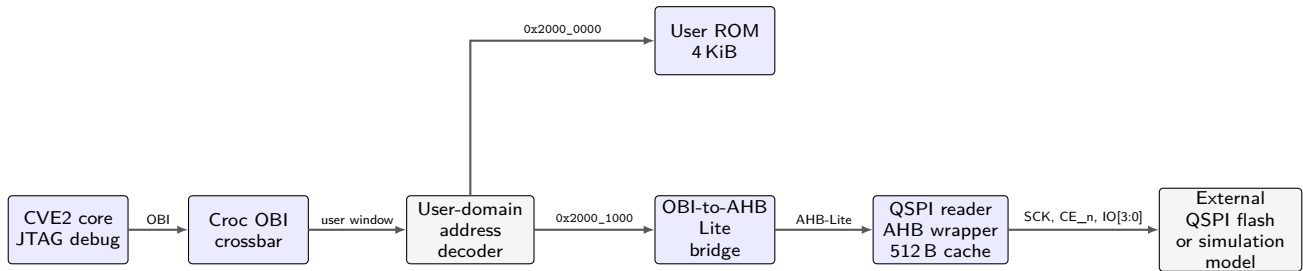
Table 1 summarizes the memory regions that matter for the project. The key design point is that the QSPI flash byte address zero is intentionally aligned with CPU-visible address 0x2000_1000. The bridge subtracts this offset before issuing the AHB-LITE request to the controller, so CPU address 0x2000_1000 + *n* accesses flash byte offset *n*.

Table 1: Relevant address regions used by the final design. The end column uses inclusive byte addresses.

Start	End	Function
0x0200_0000	0x0200_3FFF	Boot ROM. Waits in WFI until JTAG wakes the core, then jumps to the XIP entry point.
0x0204_0000	0x0207_FFFF	CLINT. Used by the testbench through JTAG to assert <code>msip</code> .
0x1000_0000	0x1000_0FFF	On-chip SRAM. Holds writable data and stack during XIP execution.
0x2000_0000	0x2000_0FFF	User ROM. Returns the zero-terminated group string.
0x2000_1000	0x2100_0FFF	16 MiB QSPI/XIP mapping. Firmware text and read-only data are linked here.
other user addresses	-	User-domain OBI error subordinate returning 0xBADCAB1E.

2.2 Read Path Through the New Subsystem

Figure 2 shows the data path added for flash accesses. Note that the controller is subordinate-only: it does not initiate transactions into the rest of the SoC. The bridge is therefore the only new protocol conversion stage between Croc and the external flash logic.

**Figure 2:** Detailed flash-read path inside the modified user domain.

2.3 Pin Reuse Strategy and QSPI Pad Assignment

The external QSPI interface is implemented by reusing the upper six GPIO pads. No additional chip-level pads were added. GPIO26 drives the serial clock, GPIO27 drives the active-low chip-enable signal, and GPIO28–GPIO31 form the four bidirectional QSPI data lines. Six GPIO pads were used because the QSPI interface requires six connections. The four unused pads alone would not have been sufficient and using them together with additional GPIO pads would have resulted in a mix of different current ratings, which could have caused issues during final hardware testing.

Table 2: Logical GPIO pad assignment for the external QSPI interface.

GPIO pad	QSPI signal	Direction
GPIO26	<code>qspi_sck_o</code>	output
GPIO27	<code>qspi_ce_n_o</code>	output
GPIO28	QSPI IO0	bidirectional
GPIO29	QSPI IO1	bidirectional
GPIO30	QSPI IO2	bidirectional
GPIO31	QSPI IO3	bidirectional

Because these pads are no longer used as conventional GPIOs, `rtl/croc_chip.sv` ties `soc_gpio_i[31:26]` to zero on the internal GPIO input side. SCK and chip-enable are permanently configured as outputs. The four data pads are controlled by the QSPI controller output-enable vector so that they can switch between output and input phases during a flash transaction.

Figure 3 shows the QSPI signal assignment along the top pad edge. The drawing is schematic and not to scale.

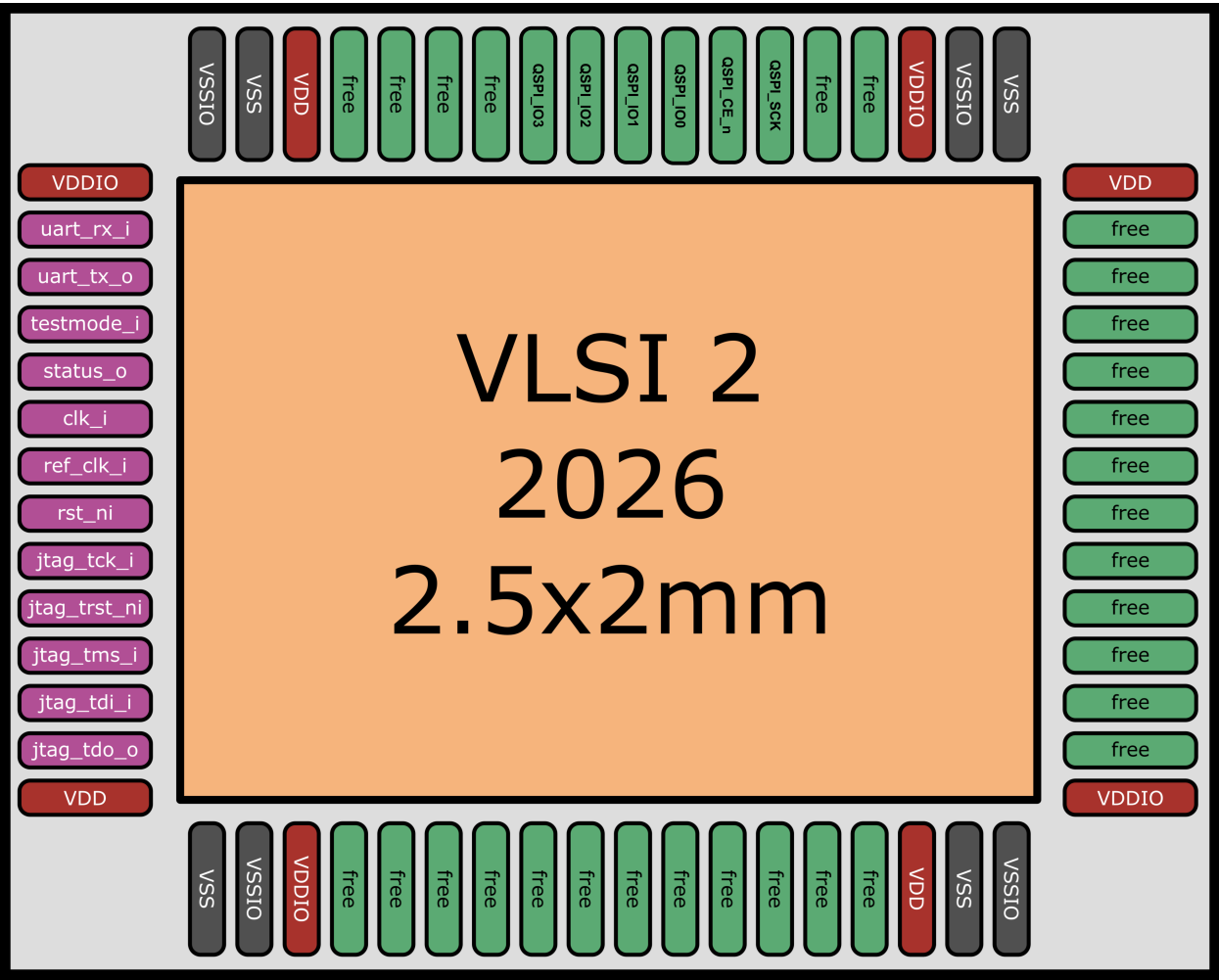


Figure 3: QSPI pad assignment along the top edge of the chip. GPIO26 drives QSPI_SCK, GPIO27 drives QSPI_CE_n, and GPIO28–GPIO31 implement the bidirectional QSPI_I00–QSPI_I03 signals.

3 RTL Modifications and Module Functionality

3.1 Modified and Added Source Files

Table 3 summarizes the main RTL, verification, software, and backend files modified for the final implementation.

Table 3: Main RTL, testbench, and script changes for the QSPI integration.

File	Status	Purpose
Bender.yml	modified	Adds the Efabless cache/controller sources, the custom bridge, and the simulation flash model to the correct build targets.
rtl/user_pkg.sv	modified	Defines <code>UserRom</code> and <code>UserQspi</code> subordinate IDs and the user-domain address map.
rtl/user_domain.sv	modified	Performs user-domain decode, hosts the hardcoded user ROM, instantiates the OBI-to-AHB bridge and the QSPI controller, and preserves the default error path.
rtl/user_domain/qspi/obi_to_ahbl.sv	added	Converts one outstanding OBI read request into the minimal AHB-Lite transaction format expected by the controller.
rtl/user_domain/qspi/efabless/DMC.v	added	Provides the direct-mapped read-only cache storage.
rtl/user_domain/qspi/efabless/EF_QSPI_XIP_CTRL.v	added	Implements the QSPI line reader and software-reset logic for continuous 0xEB reads.
rtl/user_domain/qspi/efabless/EF_QSPI_XIP_CTRL_AHB_L.v	added	Wraps the flash reader behind an AHB-Lite subordinate view and inserts cache-hit / cache-miss stalling.
rtl/croc_soc.sv	modified	Propagates the new QSPI signals between the top-level SoC and user domain.
rtl/croc_chip.sv	modified	Maps QSPI signals to GPIO26–GPIO31 and disconnects those bits from normal GPIO input capture.
rtl/bootrom/bootrom.S	modified	Changes the boot target to 0x2000_1000 and points the trap-handler vector table into the XIP region.
rtl/bootrom/bootrom.sv	modified	Regenerated Boot ROM image corresponding to the modified assembly file.
rtl/test/tb_croc_soc.sv	modified	Instantiates the QSPI flash model, passes the program image via <code>+flash=...</code> , and wakes the core through CLINT.
rtl/test/qspi_flash_model.sv	added	Provides a 16 MiB byte-addressable flash model for Verilator.
rtl/test/croc_vip.sv	modified	Keeps the existing JTAG/UART infrastructure but no longer uses JTAG program loading.
verilator/run_verilator.sh	modified	Changes <code>-run</code> semantics so it launches a <code>.flash.hex</code> image rather than a JTAG-loaded SRAM image.
verilator/croc.f	modified	Includes the new QSPI-related simulation sources.
sw/link.ld	modified	Places vectors, startup, text, and rodata in flash; places writable sections in SRAM.
sw/Makefile	modified	Generates <code>.bin</code> and byte-per-line <code>.flash.hex</code> images for the flash model.
sw/helloworld.c	modified	Provides a simple direct-QSPI boot smoke program.
sw/test/test_qspi_sample_reads.c	added	Reads fixed offsets across the mapped flash window and prints the observed values.
sw/test/test_qspi_vs_sram_read.c	added	Shows the user ROM string, the first XIP words, and the fact that SRAM no longer holds program text.
openroad/scripts/02_placement.tcl	modified	Reduces placement density to improve routability.
openroad/scripts/03_cts.tcl	modified	Adds CTS sink clustering and an extra design-repair stage.
openroad/scripts/04_routing.tcl	modified	Adds more repair passes and increases detailed-routing iterations.
openroad/src/constraints.sdc	modified	Holds the backend timing constraints used by the archived run.

3.2 Changes Relative to the Reference Flow

Table 4 summarizes the functional changes introduced by the project compared with the unmodified reference flow.

Table 4: Summary of changes relative to the Croc reference flow.

Area	Reference flow	QSPI project change
User domain	All user-domain accesses terminate at the OBI error responder.	Added address decoding, user ROM, QSPI bridge, QSPI controller, cache, and fallback error path.
Address map	User range is reserved but not used for a peripheral.	Added user ROM at 0x2000_0000 and 16 MiB QSPI/XIP window starting at 0x2000_1000.
Bus interface	Only Croc OBI is present in the user domain.	Added a read-only OBI-to-AHB-Lite bridge for the Efabless controller.
QSPI controller	No external-flash logic exists.	Added QSPI controller, AHB-Lite wrapper, and 512 B direct-mapped cache.
Write handling	No QSPI target exists.	Writes to the XIP region return an OBI error response and do not start a flash transaction.
Pad mapping	GPIO26–GPIO31 are standard GPIO pads.	GPIO26 is QSPI SCK, GPIO27 is QSPI CE_n, and GPIO28–GPIO31 are QSPI IO[3:0].
Boot flow	Reference boot flow loads or starts software from the original configuration.	Boot ROM jumps directly to 0x2000_1000 after CLINT wake-up.
Software placement	Program code is linked for the original SRAM-based flow.	Vectors, startup code, text, and read-only data are linked into the XIP window; writable data and stack remain in SRAM.
Simulation	JTAG-oriented program-loading flow.	Added QSPI flash model and <code>+flash=<image></code> execution flow using byte-per-line <code>.flash.hex</code> images.
Verification	Baseline peripheral tests only.	Added QSPI sample-read and XIP-versus-SRAM software tests.
Backend flow	Reference placement, CTS, and routing settings.	Reduced placement density, added CTS repair steps, and increased routing repair/detail iterations.

3.3 Address Decode and User-Domain Structure

The address-map update is implemented in `rtl/user_pkg.sv`. The enum `user_demux_outputs_e` introduces two explicit subordinate IDs:

- `UserRom = 1`
- `UserQspi = 2`

with `UserError = 0` reserved as the default target.

The corresponding address rules define:

- `UserBaseAddr .. UserBaseAddr + 0x0000_0FFF` for the user ROM;
- `UserBaseAddr + 0x0000_1000 .. UserBaseAddr + 0x0100_0FFF` for the 16 MiB QSPI window.

Inside `rtl/user_domain.sv`, the incoming user-domain OBI request is decoded with `addr_decode` and distributed through `obi_demux`. The design therefore preserves normal Croc interconnect behavior: there is still one request ingress, but it now fans out internally to three destination blocks.

3.4 User ROM

The user ROM is intentionally small and fixed. It returns a zero-terminated ASCII string containing the two group names. It is implemented as a local function `automatic logic [31:0] user_rom_word(...)` with a word-aligned address lookup. Although simple, this block serves two purposes:

1. it satisfies the expected project checker convention at `0x2000_0000`;
2. it provides a useful low-risk directed-read test path through the modified decode logic.

The ROM responds in one cycle after request capture. Reads return the stored word, while writes return `0xBADCAB1E` and set the OBI error bit.

Table 5: Expected user-ROM words returned by the implementation.

Address	Value	ASCII interpretation
0x2000_0000	0x206E614A	"Jan "
0x2000_0004	0x75656148	"Haeu"
0x2000_0008	0x72657373	"sser"
0x2000_000C	0x6E6E616D	"mann"
0x2000_0010	0x694E202C	", Ni"
0x2000_0014	0x53206F6E	"no S"
0x2000_0018	0x65676E69	"inge"
0x2000_001C	0x00000072	"r\0\0\0"

3.5 OBI-to-AHB-Lite Bridge

The new bridge in `rtl/user_domain/qspi/obi_to_ahbl.sv` exists because the Efabless controller presents a reduced AHB-Lite subordinate interface rather than an OBI interface. The bridge is intentionally minimal:

- only one outstanding request is supported;
- all writes are rejected with an OBI error response;
- the incoming address is translated by subtracting `AddrOffset = 0x2000_1000`;
- `HREADYOUT` from the controller is reused as the global `HREADY` input, so the bridge naturally stalls on cache misses.

The internal FSM uses three states:

Idle Wait for an OBI request. If the subordinate is ready and the request is a read, capture `aid` and address and launch an AHB-LITE transfer.

AhbWait Hold address/control stable while the controller stalls the bus on a miss. When `HREADYOUT` rises, capture `HRDATA`.

ObiResp Return the stored data and the original ID as a one-cycle OBI response.

A write request is accepted only in the sense that it consumes the request and produces a legal OBI response; no external flash transaction is issued.

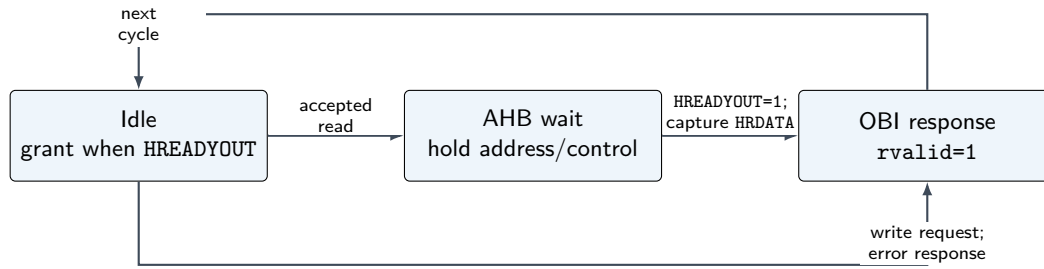


Figure 4: Bridge behavior for one request at a time.

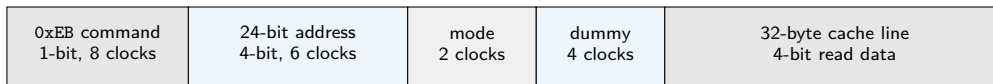
3.6 QSPI Controller, Cache, and Serial Transfer

The design reuses the Efabless `EF_QSPI_XIP_CTRL_AHBL.v` wrapper together with `EF_QSPI_XIP_CTRL.v` and `DMC.v`. In this project, the instantiation is configured with:

- `NUM_LINES` = 16
- `LINE_SIZE` = 32
- `RESET_CYCLES` = 999

This produces a direct-mapped cache of 16×32 bytes = 512 bytes total. On a hit, the requested 32-bit word is returned immediately from cache storage. On a miss, the controller stalls the AHB-LITE side and fetches one full aligned 32-byte cache line from flash. The cache line is then written into `DMC.v`, the matching word is returned, and the transaction completes.

The flash-reader protocol used by the model and by the controller is Quad-I/O Fast Read with opcode `0xEB`. The first transaction sends command, address, mode, and dummy phases. Continuous transfers omit the repeated opcode and resume directly with address, mode, dummy, and data.



Continuous read omits the opcode on subsequent transactions.

Figure 5: Flash-read transaction structure implemented by the integrated controller and the simulation flash model.

3.7 Boot, Linker, and Software Memory Placement

The software boot path is modified in two places:

1. the Boot ROM now jumps directly to `0x2000_1000` after wake-up instead of reading a boot address from `soc_ctrl`;
2. the linker script places executable content in the XIP region and writable content in SRAM.

The Boot ROM trap wrapper also changes the vector-table base to the XIP region, so the following locations are used:

- `0x2000_1000`: `jal x0, _start`
- `0x2000_1004`: exception handler pointer
- `0x2000_1008`: interrupt handler pointer

This means the startup stub in `sw/crt0.S` remains largely unchanged in structure, but it is no longer placed in SRAM; instead, the linker places `.vectors`, `.text._start`, `.text`, and `.rodata` into flash at `0x2000_1000`.

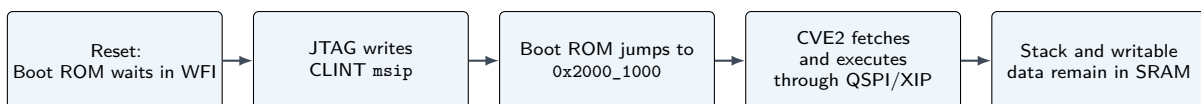


Figure 6: Boot flow after the Boot-ROM and linker changes.

3.8 Key Source-Code Locations

The following locations identify the main implementation changes:

- `rtl/user_pkg.sv`, lines 28–51: user-domain subordinate IDs and address rules.
- `rtl/user_domain.sv`, lines 89–121: user-domain decode and OBI demultiplexing.
- `rtl/user_domain.sv`, lines 138–180: user ROM implementation.
- `rtl/user_domain.sv`, lines 183–240: bridge and QSPI controller instantiation.
- `rtl/user_domain/qspi/obi_to_ahbl.sv`, lines 38–131: bridge FSM and transaction conversion.

- rtl/croc_soc.sv, lines 28–35 and 111–116: user-domain QSPI ports at SoC level.
- rtl/croc_chip.sv, lines 75–87 and 145–150: GPIO repurposing for QSPI pads.
- rtl/bootrom/bootrom.S, lines 75–80 and 121–143: direct XIP jump and XIP trap dispatch.
- sw/link.ld, lines 22–83: final memory layout for flash and SRAM.
- rtl/test/tb_croc_soc.sv, lines 46–54 and 149–174: flash-model hookup and CLINT wake-up sequence.
- rtl/test/qspi_flash_model.sv, lines 33–120: flash-image loading and protocol handling.

4 Verification Strategy and Test Plan

The repository contains an end-to-end simulation environment in which JTAG is still used for debug control, but no longer for code loading. Instead, the Verilator testbench instantiates a QSPI flash model, initializes it from a `.flash.hex` file, wakes the processor by writing to CLINT through JTAG, and then waits for the Boot ROM and application to finish. This setup matches the intended system behavior much more closely than the original SRAM-only reference-flow simulation.

4.1 Verification Environment

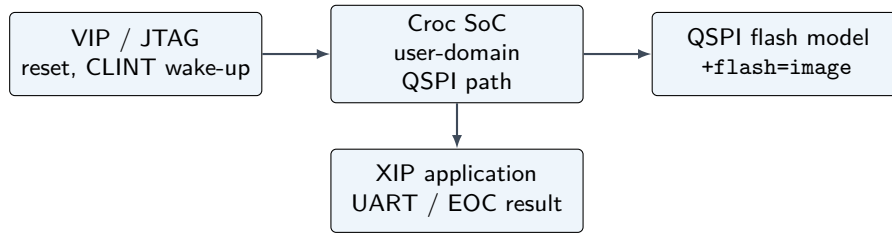


Figure 7: Verification environment used by the modified Verilator flow.

4.2 Verification Objectives

The final verification focuses on the following behavior:

1. the user-domain decode selects the ROM and QSPI regions correctly;
2. the user ROM returns the intended group string;
3. the bridge performs read transactions correctly between the Croc OBI fabric and the controller AHB-LITE interface;
4. the QSPI controller performs a cache-line read from the flash model;
5. the CPU boots and executes software linked into the XIP region;
6. sampled XIP words match the generated flash image across the mapped address range.

4.3 Test Matrix

Table 6: Final functional tests and supporting evidence.

ID	Stimulus / setup	Observed result and evidence
T1	Boot <code>sw/bin/test/test_qspi_sample_reads.flash.hex</code> and <code>sw/bin/test/test_qspi_vs_sram_read.flash.hex</code> .	Both applications reach Simulation finished: SUCCESS ; see Listing 2.
T2	Run <code>sw/bin/test/test_qspi_sample_reads.flash.hex</code> .	All 11 sampled words match the generated flash image; see Table 7 and Listing 2.
T3	Run <code>sw/bin/test/test_qspi_vs_sram_read.flash.hex</code> .	The User ROM string is correct, XIP words are non-zero, and sampled SRAM words are zero; see Listing 2.
T4	Read <code>0x2000_1000</code> , <code>0x2000_1004</code> , and <code>0x2000_1010</code> , which belong to one 32-byte cache line.	Correct words are returned; see Table 7. The QSPI cache-line fill is shown in Figure 8.
T5	Read distributed addresses across the 16 MiB XIP mapping.	All 11 sampled words match the generated flash image at the listed XIP addresses; see Table 7.

4.4 Expected and Observed XIP Values

The generated `test_qspi_sample_reads.flash.hex` image determines the expected values for the sample-read regression. Table 7 compares those expected values with the observed words returned through the XIP mapping. The

flash model initializes the unused portion of its 16 MiB array to zero before `$readmemh` overlays the program image.

Table 7: QSPI/XIP words observed in simulation compared with the flash-image contents.

QSPI address	Expected word	Observed word	Result
0x20001000	0x00C0006F	0x00C0006F	PASS
0x20001004	0x2000111C	0x2000111C	PASS
0x20001010	0xFF410113	0xFF410113	PASS
0x20001100	0x01010113	0x01010113	PASS
0x20002000	0x00000000	0x00000000	PASS
0x20011000	0x00000000	0x00000000	PASS
0x20101000	0x00000000	0x00000000	PASS
0x20401000	0x00000000	0x00000000	PASS
0x20801000	0x00000000	0x00000000	PASS
0x20C01000	0x00000000	0x00000000	PASS
0x2100FFC	0x00000000	0x00000000	PASS

For `test_qspi_vs_sram_read.flash.hex`, the first words in the XIP window are:

0x00C0006F, 0x20001020, 0x20001024, 0xF0000117

which is consistent with the vector table and flash-linked startup sequence generated by the supplied software build.

4.5 Waveform Evidence

The QSPI waveform in Figure 8 was captured from `verilator/croc.fst`. It shows the bridge request, controller stall, and serial flash transaction during a cache-line fill.

- top-level serial pins: `qspi_ce_n`, `qspi_sck`, `qspi_dout[3:0]`, `qspi_douten[3:0]`, `qspi_din[3:0]`;
- bridge-level handshaking: `obi_req_i.req`, `obi_rsp_o.gnt`, `obi_rsp_o.rvalid`, `hsel_o`, `htrans_o`, `hreadyout_i`, `hrdata_i`;
- controller/cache behavior: cache hit signal, line-fill trigger, and returned word if visible in the chosen hierarchy.

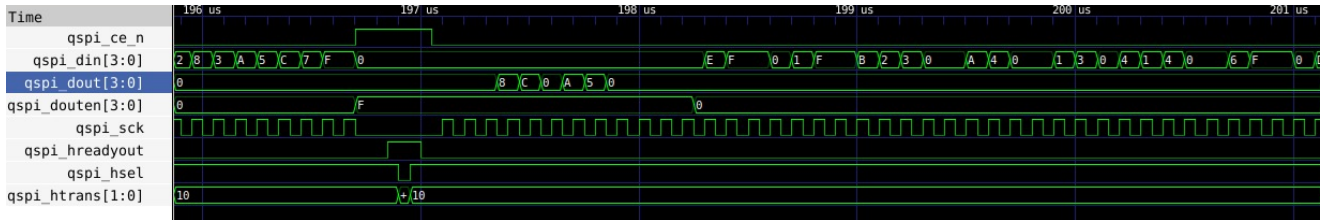


Figure 8: Observed QSPI cache-line fill in Verilator. The bridge launches an AHB-Lite non-sequential transfer through `qspi_hsel` and `qspi_htrans`. The controller stalls the request through `qspi_hreadyout` while `qspi_ce_n`, `qspi_sck`, `qspi_dout`, `qspi_douten`, and `qspi_din` show the serial QSPI transaction.

The waveform demonstrates the miss-side transaction: the bridge issues an AHB-Lite request, the controller stalls the request while it fetches a cache line, and the serial command, address, and data phases are visible on the QSPI signals. The sample-read software accesses `0x2000_1000`, `0x2000_1004`, and `0x2000_1010`, which all belong to the same 32-byte cache line. It also accesses addresses separated by large offsets across the complete 16 MiB XIP range. Together, the waveform and successful read results demonstrate a QSPI cache-line fill and correct data delivery across the XIP address range.

5 Simulation Bring-Up, Software Tests, and Verification Results

5.1 Simulation Flow

The intended software and simulation flow differs from the reference flow, and the required build sequence is already encoded in the repository. The commands below are sufficient to regenerate the software, re-create the Verilator file list, build the simulator, and execute a selected flash image. It is important to follow the run-script semantics: use the `.flash.hex` image, not the `.hex` image, to properly load the program into the QSPI flash simulation model.

Listing 1: Primary software and Verilator flow used by the project.

```

oseda bash
cd sw
make clean all

cd ../verilator
./run_verilator.sh --flist
./run_verilator.sh --build
./run_verilator.sh --run ../sw/bin/test/test_qspi_sample_reads.flash.hex
./run_verilator.sh --run ../sw/bin/test/test_qspi_vs_sram_read.flash.hex

```

In the modified project, `--run` no longer means “load this file into SRAM through JTAG”. Instead, it launches the simulator with `+flash=<path>` so that the `qspi_flash_model.sv` instance loads the selected byte-per-line image into its internal 16 MiB memory array.

5.2 Software Used to Exercise the Hardware

Two dedicated software programs are used to exercise the hardware:

`sw/test/test_qspi_sample_reads.c` reads fixed offsets across the mapped flash range. This validates the basic QSPI/XIP data path because the expected values can be derived from the generated image.

`sw/test/test_qspi_vs_sram_read.c` prints the user ROM string, then compares the first XIP words against SRAM words to demonstrate that the program is linked in flash while writable data and stack remain in SRAM.

The generated `.flash.hex` files under `sw/bin/test/` are the software images used for the reported regressions.

5.3 Simulation Evidence

The final verification evidence consists of the dedicated QSPI regressions `test_qspi_sample_reads.flash.hex` and `test_qspi_vs_sram_read.flash.hex`. Both simulations completed with **Simulation finished: SUCCESS**. The sample-read program compared eleven addresses across the 16 MiB XIP window with the generated flash image; all observed words matched. The XIP-versus-SRAM program verified the User ROM string, observed valid program data in the XIP window, and observed zero values at the sampled SRAM locations.

5.4 Functional Verification Results

Table 8: Functional verification summary.

Test	Result / observed behavior	Evidence
T1	PASS — Both dedicated QSPI/XIP applications booted through the modified Boot ROM and reached Simulation finished: SUCCESS .	Listing 2.
T2	PASS — All 11 sampled QSPI words matched the generated <code>.flash.hex</code> image.	Table 7 and Listing 2.
T3	PASS — The User ROM string was read correctly; XIP words were non-zero while the sampled SRAM words were zero.	Listing 2.
T4	PASS — Reads at <code>0x2000_1000</code> , <code>0x2000_1004</code> , and <code>0x2000_1010</code> access the same 32-byte cache line. Correct values were returned, and the QSPI line-fill transaction is shown in the waveform.	Table 7 and Figure 8.
T5	PASS — All 11 sampled words matched the generated flash image at addresses distributed across the 16 MiB XIP range.	Table 7.

Listing 2: Selected QSPI/XIP Verilator results.

```

QSPI sample-read test:
User ROM : 0x20000000 .. 0x20000fff
QSPI/XIP : 0x20001000 .. 0x21000fff
QSPI[0x20001000] = 0x00c0006f
QSPI[0x20001004] = 0x2000111c
QSPI[0x20001010] = 0xff410113
QSPI sample read test finished.
Simulation finished: SUCCESS

QSPI/XIP versus SRAM test:
Hello from QSPI XIP!
XIP[0x20001000] = 0x00c0006f
XIP[0x20001004] = 0x20001020
SRAM[0x10000000] = 0x00000000
SRAM[0x10000004] = 0x00000000
The code is no longer located at SRAM 0x10000000!
Simulation finished: SUCCESS

```

6 Backend Implementation, Physical Results, and Signoff Status

6.1 Synthesis and Backend Flow

The implementation flow follows the reference-flow structure:

1. RTL file-list generation with Bender;
2. synthesis with Yosys;
3. floorplan, placement, CTS, routing, and finishing in OpenROAD;
4. GDS generation and design-rule checks with KLayout.

The core commands already available in the repository are:

Listing 3: Primary backend command sequence.

```
cd yosys
./run_synthesis.sh --flist
./run_synthesis.sh --synth

cd ../openroad
./run_backend.sh --all

cd ../klayout
./def2gds-croc
./run_drc-croc
```

6.2 Backend Script Changes

Three backend scripts were adjusted for this design revision:

- openroad/scripts/02_placement.tcl: global-placement density was reduced from 0.60 to 0.55 in both placement calls.
- openroad/scripts/03_cts.tcl: sink clustering parameters were added for clock-tree synthesis, followed by repair_clock_nets and a repair_design step after CTS.
- openroad/scripts/04_routing.tcl: extra design-repair calls were inserted, global-routing parasitics were re-estimated repeatedly, and the detailed-routing end-iteration count was increased from 20 to 40.

These changes are consistent with a design that became harder to route and close after the user-domain logic and pad repurposing were introduced.

6.3 Physical Metrics and Signoff Results

The QSPI-integrated design was synthesized, placed, routed, converted to GDS, and checked with the available physical-verification flow. The final synthesized cell area is 1 636 941.1104 μm^2 . The final routed implementation targets a 10.0 ns clock period, corresponding to 100 MHz.

Calibre LVS completed successfully. The final LVS report gives the overall comparison result **CORRECT**. After KLayout, DRC reported only **metal1_pin_Offgrid** markers located inside the supplied SRAM and pad macro GDS views. The course FAQ identifies these as known macro-library artifacts rather than physical violations introduced by the project. They are therefore waived for this report.

Table 9: Final backend implementation and verification status.

Metric	Final result / status
Target clock period	10.0 ns
Target frequency	100 MHz
Top-level synthesized cell area	1 636 941.1104 μm^2
Setup WNS	−0.64 ns
Setup TNS	−726.15 ns
Setup violation count	2758
Hold violation count	0
Maximum slew violations	189 (caused by reference design and strict maximum-limit settings)
Maximum fanout violations	2 (caused by reference design and strict maximum-limit settings)
Maximum capacitance violations	83 (caused by reference design and strict maximum-limit settings)
Calibre LVS	PASS — overall comparison result CORRECT
KLayout DRC	PASS with FAQ waiver — only known metal1_pin_Offgrid markers inside supplied SRAM/pad macros
OpenROAD route DRC	2 Metal1 spacing violations that were fixed after KLayout
Power	Not measured

6.4 Physical-Implementation Discussion

The project was successfully taken through synthesis, placement, CTS, routing, GDS generation, KLayout DRC, and Calibre LVS. Functional simulation and Calibre LVS passed, and KLayout DRC passed under the documented macro-library waiver.

The implementation is not closed at the 100 MHz timing target. The routed report contains negative setup slack and 2758 setup violations. Hold timing is clean, but maximum-slew, fanout, and capacitance checks still report 189, 2, and 83 violations respectively. These violations do not indicate functional failures of the QSPI subsystem. Similar issues are also observed in the reference flow and result from strict maximum-limit constraints. As discussed with the supervisors, they are considered expected and are not problematic.

The KLayout DRC result is treated as passing because the remaining `metal1_pin_Offgrid` markers occur inside the supplied SRAM and pad macros. The course FAQ explicitly permits these known macro-library artifacts to be ignored. The two Metal1 spacing markers reported by OpenROAD were fixed with KLayout.

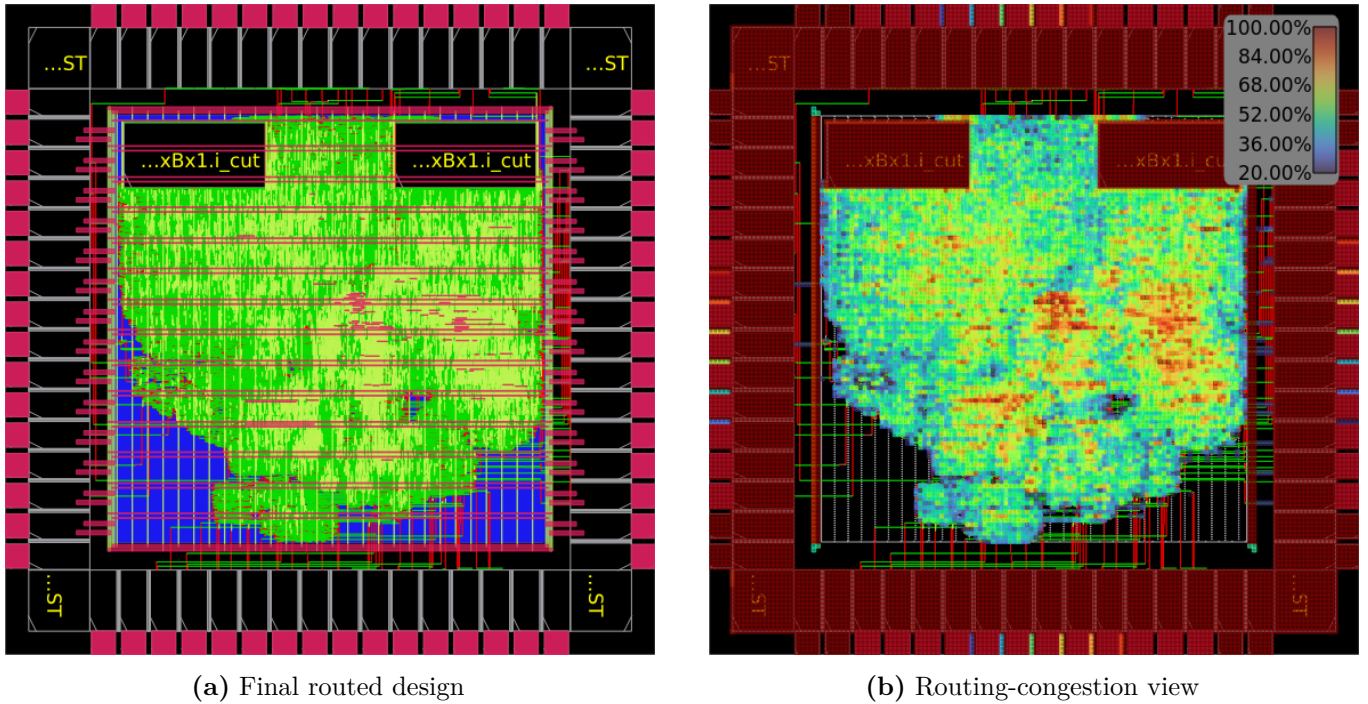


Figure 9: Final physical implementation of the QSPI-integrated Croc design: (a) routed layout and (b) routing-congestion view.

The QSPI-integrated design was successfully synthesized, routed, converted to GDS, functionally verified in Verilator, and matched by Calibre LVS. KLayout DRC passes with the documented SRAM/pad macro off-grid waiver. Remaining OpenROAD implementation limitations are setup timing, maximum slew, fanout and capacitance; These are expected and were confirmed by the supervisors via email as not being problematic and not originating from the QSPI extension.

6.5 Problems Encountered and How They Were Addressed

The implementation-related challenges can be summarized as follows:

1. **Protocol mismatch.** Croc uses OBI, while the selected controller expects a minimal AHB-LITE subordinate interface. This was resolved with a custom, read-only bridge.
2. **Boot-path mismatch.** The reference-flow simulation expects software to be loaded into SRAM through JTAG. For XIP, the boot path was changed so the Boot ROM jumps directly into the flash-mapped window and the simulator loads a flash image into a dedicated model.
3. **Address-space partitioning.** The user ROM required by the project checker had to coexist with the flash window. This was handled by reserving `0x2000_0000..0x2000_0FFF` for ROM and starting QSPI at `0x2000_1000`.
4. **Pad reuse.** GPIO26–GPIO31 were reassigned as QSPI pins, which required wrapper-level changes and explicit input-side disabling for those GPIO bits.
5. **Backend closure pressure.** Placement density and routing strategy had to be relaxed and repair stages strengthened.

7 Reproducibility and Conclusion

7.1 Reproducibility Notes

The design state described in this report can be reproduced with the following steps:

1. source the project environment through `env.sh`;
2. rebuild software with `make all`;
3. regenerate the Verilator file list and rebuild the simulator if sources changed;
4. run the selected `.flash.hex` images through the modified `run_verilator.sh` like shown in subsection 5.1;
5. inspect `verilator/croc.log` and `verilator/croc.fst`;
6. run synthesis and backend through the provided scripts;
7. generate GDS and rerun DRC/LVS with the team's final command sequence.

7.2 Conclusion

The project transforms the Croc user domain into a memory-mapped QSPI execute-in-place subsystem. The final implementation adds user-domain address decoding, a user ROM, a read-only OBI-to-AHB-Lite bridge, a cached QSPI reader, GPIO pad routing, direct Boot-ROM hand-off to the XIP window, flash-image generation, and dedicated simulation tests.

The dedicated QSPI regressions passed in Verilator. All eleven sampled XIP words matched the generated flash image, and the XIP-versus-SRAM program confirmed that executable code is fetched from `0x2000_1000` while the sampled SRAM locations remain empty. The waveform demonstrates the serial QSPI line-fill sequence and the bridge stall/response behavior.

The physical flow produced a routed layout and final GDS. Calibre LVS passed with overall result **CORRECT**. KLayout DRC passes with the documented waiver for known `metal1_pin_offgrid` markers in supplied SRAM and pad macros. The remaining work consists of evaluating the maximum achievable clock frequency on physical hardware, since the extension introduces a setup WNS of -0.64 ns when targeting 100 MHz. Setup timing, maximum slew, fanout, capacitance violations are considered normal and expected according to the course guidelines and supervisors.